DBMS Project Report

AptManager: Multi-Complex Apartment Management and Analytics System

DBMS Experiential Learning, Level 3 Project · PES University

---

1. Project Title

Multi-Complex Apartment Management and Analytics System (AptManager)

GitHub repository: https://github.com/Kundhan73/DBMS_Project

2. Student SRNs and Names

Name	SRN
Kundhan Vudutha	PES1UG24AM327
Tejas Shetty	PES1UG24AM305

3. Abstract

Property management firms in Bengaluru usually run several apartment complexes at once, with each flat having owners, tenants, leases, parking slots and monthly rent, maintenance and parking bills. In practice this information lives in spreadsheets, WhatsApp groups and paper registers, which leads to overlapping leases, double-booked parking slots, no clear view of overdue rent, and no audit trail of who changed what.

AptManager replaces these registers with a single database-driven web application that enforces business rules inside the database itself. PostgreSQL 16 holds the relational data in a normalised (3NF) schema of 19 tables, with CHECK and EXCLUDE constraints, 21 triggers, stored procedures and functions for idempotent monthly billing, late fees and lease termination, and 7 views plus a materialized view for analytics. A second paradigm, a vector database built on pgvector, stores 384-dimensional sentence embeddings of maintenance requests, so staff can search complaints by meaning ("water dripping from

the roof" finds "seepage from the bathroom ceiling"), find similar past requests, and detect duplicates. Keyword full-text search and vector search are combined into hybrid search using Reciprocal Rank Fusion.

The web interface is built with Next.js 16 and talks to the database only through an API layer. It provides four roles (Admin, Manager, Owner, Tenant) with row-level scoping, dashboards for occupancy, billed vs collected revenue, dues ageing and maintenance SLA compliance, nine SQL reports that display their own SQL, and an audit log written by triggers. The system is verified by 75 automated tests (database, vector and API suites).

4. Technology Stack

Layer	Choice
Relational database	PostgreSQL 16
Vector database	pgvector (>= 0.5) inside PostgreSQL, HNSW index, cosine distance
Other extensions	`btree_gist` (EXCLUDE constraints), `pg_trgm` (typo-tolerant name search)
Full-text search	PostgreSQL `tsvector` + GIN index
Embeddings	`Xenova/all-MiniLM-L6-v2` (384-d) via `@huggingface/transformers`, run locally in Node.js
Backend	Next.js Route Handlers (`src/app/api`), `pg`, `zod`, `jose` (JWT), `bcryptjs`
Frontend	Next.js 16 App Router, React 19, Tailwind CSS 4, Recharts
Runtime	Node.js 20+
Testing	Vitest (database, vector and API suites)
DevOps	Docker Compose (`pgvector/pgvector:pg16`), Neon (hosted PostgreSQL) and Render for demo hosting

Why this split of databases: structured, related, money-bearing data (units, leases, invoices, payments) needs ACID transactions, foreign keys and constraints, so it lives in PostgreSQL. Free-text maintenance complaints need meaning-based retrieval, so they are embedded and stored in pgvector. Keeping the vectors in the same database gives real foreign keys with `ON DELETE CASCADE`, one-transaction inserts of a request and its embedding, and single queries that combine vector distance with relational access filters.

5. SQL – Database Commands

All DDL lives in `db/` and is applied in order by `npm run db:setup`. The statements below are taken from the project's schema (as documented in its generated data dictionary); see `db/01_schema.sql` for the exact source.

5.1 Extensions and domains (DDL)

```
CREATE EXTENSION IF NOT EXISTS vector;
CREATE EXTENSION IF NOT EXISTS btree_gist;
CREATE EXTENSION IF NOT EXISTS pg_trgm;

CREATE DOMAIN email_address AS varchar(120);   -- with case-insensitive email
pattern CHECK
CREATE DOMAIN phone_in      AS varchar(10)  CHECK (VALUE ~ '^[6-9][0-9]
{9}$');
CREATE DOMAIN money_inr     AS numeric(12,2) CHECK (VALUE >= 0);
```

5.2 CREATE TABLE (key tables)

```
CREATE TABLE app_user (
  user_id       bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  email         email_address NOT NULL,
  password_hash text NOT NULL,
  full_name     varchar(100) NOT NULL,
  phone         phone_in,
  role          varchar(10) NOT NULL,
  is_active     boolean NOT NULL DEFAULT true,
  created_at    timestamptz NOT NULL DEFAULT now(),
  last_login_at timestamptz,
  CONSTRAINT chk_password_bcrypt CHECK (password_hash ~ '^\$2[aby]\$[0-9]
{2}\$[./A-Za-z0-9]{53}$'),
  CONSTRAINT chk_user_name CHECK (length(btrim(full_name)) >= 2),
```

```
  CONSTRAINT chk_user_role CHECK (role IN
('ADMIN','MANAGER','OWNER','TENANT'))
);

CREATE TABLE complex (
  complex_id       integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  name             varchar(80) NOT NULL UNIQUE,
  address_line     varchar(200) NOT NULL,
  locality         varchar(80) NOT NULL,
  city             varchar(60) NOT NULL DEFAULT 'Bengaluru',
  pincode          char(6) NOT NULL CONSTRAINT chk_complex_pincode CHECK
(pincode ~ '^[1-9][0-9]{5}$'),
  established_year smallint CONSTRAINT chk_complex_year CHECK
(established_year BETWEEN 1950 AND 2100),
  created_at       timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE block (
  block_id     integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  complex_id   integer NOT NULL REFERENCES complex(complex_id),
  name         varchar(20) NOT NULL,
  total_floors smallint NOT NULL CONSTRAINT chk_block_floors CHECK
(total_floors BETWEEN 1 AND 60),
  CONSTRAINT uq_block_name UNIQUE (complex_id, name)
);

CREATE TABLE unit (
  unit_id             integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  block_id            integer NOT NULL REFERENCES block(block_id),
  unit_number         varchar(10) NOT NULL,
  floor_no            smallint NOT NULL CONSTRAINT chk_unit_floor CHECK
(floor_no >= 0),
  unit_type           varchar(6) NOT NULL,
  carpet_area_sqft    numeric(7,1) NOT NULL CONSTRAINT chk_unit_area CHECK
(carpet_area_sqft BETWEEN 200 AND 10000),
  base_rent           money_inr NOT NULL,
  monthly_maintenance money_inr NOT NULL,
  occupancy_status    varchar(20) NOT NULL DEFAULT 'VACANT',
  CONSTRAINT chk_unit_type   CHECK (unit_type IN
('STUDIO','1BHK','2BHK','3BHK','4BHK')),
  CONSTRAINT chk_unit_status CHECK (occupancy_status IN
('VACANT','OWNER_OCCUPIED','LEASED','UNDER_MAINTENANCE')),
  CONSTRAINT uq_unit_number  UNIQUE (block_id, unit_number)
```

```
);

CREATE TABLE ownership (
  ownership_id integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  unit_id      integer NOT NULL REFERENCES unit(unit_id),
  owner_id     integer NOT NULL REFERENCES owner_profile(owner_id),
  share_pct    numeric(5,2) NOT NULL DEFAULT 100 CONSTRAINT
chk_ownership_share CHECK (share_pct > 0 AND share_pct <= 100),
  owned_from   date NOT NULL,
  owned_until  date,
  CONSTRAINT chk_ownership_dates CHECK (owned_until IS NULL OR owned_until >
owned_from),
  CONSTRAINT ex_ownership_overlap EXCLUDE USING gist
    (unit_id WITH =, owner_id WITH =, daterange(owned_from, owned_until,
'[)') WITH &&)
);

CREATE TABLE lease (
  lease_id           integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  unit_id            integer NOT NULL REFERENCES unit(unit_id),
  tenant_id          integer NOT NULL REFERENCES tenant_profile(tenant_id),
  start_date         date NOT NULL,
  end_date           date NOT NULL,
  monthly_rent       money_inr NOT NULL,
  security_deposit   money_inr NOT NULL DEFAULT 0,
  rent_due_day       smallint NOT NULL DEFAULT 5,
  status             varchar(10) NOT NULL DEFAULT 'ACTIVE',
  terminated_on      date,
  termination_reason varchar(300),
  created_by         bigint REFERENCES app_user(user_id),
  created_at         timestamptz NOT NULL DEFAULT now(),
  CONSTRAINT chk_lease_dates   CHECK (end_date > start_date),
  CONSTRAINT chk_lease_rent    CHECK (monthly_rent > 0),
  CONSTRAINT chk_lease_deposit CHECK (security_deposit <= monthly_rent * 10),
  CONSTRAINT chk_lease_due_day CHECK (rent_due_day BETWEEN 1 AND 28),
  CONSTRAINT chk_lease_status  CHECK (status IN
('PENDING','ACTIVE','ENDED','TERMINATED')),
  CONSTRAINT ex_lease_no_overlap EXCLUDE USING gist
    (unit_id WITH =, daterange(start_date, end_date, '[)') WITH &&)
    WHERE (status IN ('PENDING','ACTIVE'))
);

CREATE TABLE invoice (
```

```
  invoice_id            bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  unit_id               integer NOT NULL REFERENCES unit(unit_id),
  billed_to             bigint  NOT NULL REFERENCES app_user(user_id),
  invoice_type          varchar(12) NOT NULL,
  lease_id              integer REFERENCES lease(lease_id),
  parking_allocation_id integer REFERENCES parking_allocation(allocation_id),
  source_invoice_id     bigint  REFERENCES invoice(invoice_id),
  billing_month         date NOT NULL,
  amount                money_inr NOT NULL,
  amount_paid           money_inr NOT NULL DEFAULT 0,
  issued_on             date NOT NULL DEFAULT CURRENT_DATE,
  due_date              date NOT NULL,
  status                varchar(8) NOT NULL DEFAULT 'UNPAID',
  notes                 varchar(200),
  CONSTRAINT chk_inv_type   CHECK (invoice_type IN
('RENT','MAINTENANCE','PARKING','LATE_FEE')),
  CONSTRAINT chk_inv_status CHECK (status IN
('UNPAID','PARTIAL','PAID','VOID')),
  CONSTRAINT chk_inv_amount CHECK (amount > 0 AND amount_paid <= amount),
  CONSTRAINT chk_inv_month  CHECK (billing_month = date_trunc('month',
billing_month)::date),
  CONSTRAINT chk_inv_status_paid CHECK (status = 'VOID'
    OR (status = 'UNPAID'  AND amount_paid = 0)
    OR (status = 'PARTIAL' AND amount_paid > 0 AND amount_paid < amount)
    OR (status = 'PAID'    AND amount_paid = amount))
);

CREATE TABLE payment (
  payment_id   bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  invoice_id   bigint NOT NULL REFERENCES invoice(invoice_id),
  amount       money_inr NOT NULL CONSTRAINT chk_pay_amount CHECK (amount >
0),
  paid_at      timestamptz NOT NULL DEFAULT now(),
  method       varchar(13) NOT NULL CONSTRAINT chk_pay_method
               CHECK (method IN
('UPI','CARD','BANK_TRANSFER','CASH','CHEQUE')),
  reference_no varchar(40) NOT NULL UNIQUE,
  recorded_by  bigint REFERENCES app_user(user_id)
);

CREATE TABLE parking_allocation (
  allocation_id  integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  slot_id        integer NOT NULL REFERENCES parking_slot(slot_id),
```

```
  unit_id        integer NOT NULL REFERENCES unit(unit_id),
  vehicle_id     integer NOT NULL REFERENCES vehicle(vehicle_id),
  start_date     date NOT NULL,
  end_date       date,
  monthly_charge money_inr NOT NULL DEFAULT 0,
  CONSTRAINT chk_alloc_dates CHECK (end_date IS NULL OR end_date >
start_date),
  CONSTRAINT ex_slot_overlap    EXCLUDE USING gist (slot_id    WITH =,
daterange(start_date, end_date, '[)') WITH &&),
  CONSTRAINT ex_vehicle_overlap EXCLUDE USING gist (vehicle_id WITH =,
daterange(start_date, end_date, '[)') WITH &&)
);

CREATE TABLE maintenance_request (
  request_id       bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  unit_id          integer NOT NULL REFERENCES unit(unit_id),
  raised_by        bigint  NOT NULL REFERENCES app_user(user_id),
  category         varchar(14) NOT NULL,
  title            varchar(120) NOT NULL CONSTRAINT chk_mr_title CHECK
(length(btrim(title)) >= 5),
  description      text NOT NULL CONSTRAINT chk_mr_desc CHECK
(length(btrim(description)) >= 15),
  priority         varchar(6) NOT NULL DEFAULT 'MEDIUM',
  status           varchar(11) NOT NULL DEFAULT 'OPEN',
  vendor_id        integer REFERENCES vendor(vendor_id),
  estimated_cost   money_inr,
  actual_cost      money_inr,
  created_at       timestamptz NOT NULL DEFAULT now(),
  assigned_at      timestamptz,
  resolved_at      timestamptz,
  resolution_notes text,
  tenant_rating    smallint,
  search_tsv       tsvector GENERATED ALWAYS AS (
                     setweight(to_tsvector('english', title), 'A') ||
                     setweight(to_tsvector('english', description), 'B'))
STORED,
  CONSTRAINT chk_mr_priority CHECK (priority IN
('LOW','MEDIUM','HIGH','URGENT')),
  CONSTRAINT chk_mr_status   CHECK (status IN
('OPEN','ASSIGNED','IN_PROGRESS','RESOLVED','CLOSED','CANCELLED')),
  CONSTRAINT chk_mr_vendor   CHECK (status NOT IN
```

```
('ASSIGNED','IN_PROGRESS','RESOLVED','CLOSED') OR vendor_id IS NOT NULL),
  CONSTRAINT chk_mr_resolved CHECK ((status IN ('RESOLVED','CLOSED')) =
(resolved_at IS NOT NULL))
);
```

The remaining tables (`complex_manager`, `owner_profile`, `tenant_profile`, `parking_slot`, `vehicle`, `vendor`, `maintenance_status_history`, `audit_log`) follow the same pattern. The full list of 19 tables with every column, constraint and index is in `docs/DATA_DICTIONARY.md`.

5.3 Indexes

```
-- B-tree on foreign keys
CREATE INDEX idx_block_complex ON block(complex_id);
CREATE INDEX idx_unit_block    ON unit(block_id);
CREATE INDEX idx_lease_unit_status ON lease(unit_id, status);

-- Partial: dues screens read only open invoices
CREATE INDEX idx_invoice_open_due ON invoice(due_date) WHERE status IN
('UNPAID','PARTIAL');

-- Partial unique: makes the monthly billing run idempotent
CREATE UNIQUE INDEX uq_invoice_rent ON invoice(lease_id, billing_month)
  WHERE invoice_type = 'RENT' AND status <> 'VOID';
CREATE UNIQUE INDEX uq_invoice_maint ON invoice(unit_id, billing_month)
  WHERE invoice_type = 'MAINTENANCE' AND status <> 'VOID';

-- Expression: case-insensitive login
CREATE UNIQUE INDEX uq_app_user_email_lower ON app_user (lower(email::text));

-- GIN: full-text and typo-tolerant name search
CREATE INDEX idx_mr_search_tsv ON maintenance_request USING gin (search_tsv);
CREATE INDEX idx_app_user_name_trgm ON app_user USING gin (full_name
gin_trgm_ops);
```

5.4 ALTER and DROP (schema evolution, `06_migration_v2.sql`)

The v2 migration adds `unit.facing`, `complex.amenities` and `maintenance_request.is_common_area`, changes a default, creates and then drops a staging table, and drops a superseded view. It is written to be re-runnable.

```
ALTER TABLE unit ADD COLUMN IF NOT EXISTS facing varchar(2);
ALTER TABLE unit DROP CONSTRAINT IF EXISTS chk_unit_facing;
ALTER TABLE unit ADD CONSTRAINT chk_unit_facing
  CHECK (facing IS NULL OR facing IN ('N','S','E','W','NE','NW','SE','SW'));

ALTER TABLE complex ADD COLUMN IF NOT EXISTS amenities text[] NOT NULL
DEFAULT '{}';
ALTER TABLE maintenance_request ADD COLUMN IF NOT EXISTS is_common_area
boolean NOT NULL DEFAULT false;

-- staging table for a bulk unit import, dropped afterwards
DROP TABLE IF EXISTS stg_unit_import;
```

`99_teardown.sql` drops every object (materialized views, views, tables, functions, procedures, domains) in dependency order.

5.5 Triggers

14 trigger functions and 21 triggers enforce rules that CHECK constraints cannot express.

Trigger	Table	Purpose
`owner_profile_role`, `tenant_profile_role`, `complex_manager_role`	profile tables	Linked user must have the matching role
`unit_floor_check`	unit	Floor within the block's floor count
`ownership_share_check`	ownership	Live shares total at most 100% (row lock prevents races)
`lease_validate`, `lease_sync_unit`	lease	Only owned units can be leased; unit status follows leases
`payment_before_insert`	payment	Locks invoice, rejects overpayment and payments on void invoices

Trigger	Table	Purpose
`payment_after_change`	payment	Recomputes `amount_paid` and `status`
`payment_block_update`	payment	Payments are immutable
`invoice_guard`	invoice	Paid invoices cannot be voided; void cannot be reopened
`parking_allocation_check`	parking_allocation	Same complex; slot type matches vehicle; EV slots only for EVs
`mr_before_write`	maintenance_request	Legal status transitions only; timestamps set by the database
`mr_history`	maintenance_request	Logs every status change
`mr_invalidate_embedding`	maintenance_request	Deletes the vector when title or description changes
`audit_*`	lease, payment, ownership, unit, app_user, invoice	JSONB audit log (password hashes stripped)

The acting user is passed per transaction with `set_config('app.user_id', $1, true)` so triggers know who made each change.

5.6 Stored procedures and functions

Routine	Type	Purpose
`fn_generate_monthly_invoices(month)`	function	RENT (prorated), MAINTENANCE and PARKING invoices; idempotent via `ON CONFLICT DO NOTHING`
`fn_apply_late_fees(as_of, rate, grace)`	function	2% of balance (minimum Rs 100) on overdue invoices, one per invoice
`fn_record_payment(...)`	function	Records a payment with generated reference
`sp_terminate_lease(lease, date, reason)`	procedure	Terminates lease, voids later rent, releases parking, atomically
`fn_close_expired_leases(as_of)`	function	Nightly job: ACTIVE leases past end date become ENDED
`fn_refresh_unit_status(unit)`, `fn_recalc_invoice(invoice)`	functions	Derived status helpers
`fn_similar_requests(request, k)`, `fn_hybrid_search(...)`	SQL functions	Vector and hybrid retrieval

```
SELECT fn_generate_monthly_invoices(date_trunc('month', current_date)::date);
SELECT fn_apply_late_fees(current_date, 0.02, 5);
CALL sp_terminate_lease(42, DATE '2026-08-31', 'Tenant relocating');
```

5.7 Views

`v_user_complex_scope`, `v_unit_directory`, `v_complex_occupancy`, `v_invoice_balance`, `v_monthly_collections`, `v_maintenance_sla`, `v_vendor_performance`, and the materialized view `mv_complex_kpis`:

```
REFRESH MATERIALIZED VIEW CONCURRENTLY mv_complex_kpis;
```

5.8 Analytical queries (DML / SELECT)

Repeat defaulters (JOIN, GROUP BY, HAVING, aggregates)

```
SELECT ib.billed_to_name AS resident, ib.complex_name AS complex,
       count(*) AS overdue_invoices, sum(ib.balance) AS outstanding,
       max(ib.days_overdue) AS oldest_days_overdue
  FROM v_invoice_balance ib
 WHERE ib.complex_id = ANY($1) AND ib.days_overdue > 0
 GROUP BY ib.billed_to, ib.billed_to_name, ib.complex_name
HAVING count(*) >= 2
 ORDER BY outstanding DESC;
```

Leases below market rent (correlated subquery)

```
SELECT d.complex_name, d.block_name || '-' || d.unit_number AS unit,
d.unit_type, l.monthly_rent
  FROM lease l JOIN v_unit_directory d ON d.unit_id = l.unit_id
 WHERE l.status = 'ACTIVE'
   AND l.monthly_rent < 0.9 * (SELECT avg(l2.monthly_rent)
                                 FROM lease l2 JOIN v_unit_directory d2 ON
d2.unit_id = l2.unit_id
                                WHERE l2.status = 'ACTIVE'
                                  AND d2.complex_id = d.complex_id AND
d2.unit_type = d.unit_type);
```

Leases expiring with no renewal lined up (NOT EXISTS)

```
SELECT d.complex_name, d.unit_number, au.full_name, l.end_date, l.end_date -
current_date AS days_left
  FROM lease l JOIN tenant_profile t USING (tenant_id) JOIN app_user au ON
au.user_id = t.user_id
  JOIN v_unit_directory d ON d.unit_id = l.unit_id
 WHERE l.status = 'ACTIVE' AND l.end_date <= current_date + 60
   AND NOT EXISTS (SELECT 1 FROM lease nxt WHERE nxt.unit_id = l.unit_id AND
nxt.status = 'PENDING')
 ORDER BY l.end_date;
```

Revenue with subtotals (ROLLUP, GROUPING)

```
SELECT CASE WHEN GROUPING(c.name) = 1 THEN 'ALL COMPLEXES' ELSE c.name END AS
complex,
       CASE WHEN GROUPING(i.invoice_type) = 1 THEN 'SUBTOTAL' ELSE
i.invoice_type END AS invoice_type,
       sum(i.amount) AS billed, sum(i.amount_paid) AS collected
  FROM invoice i JOIN unit u USING (unit_id) JOIN block b USING (block_id)
JOIN complex c USING (complex_id)
 WHERE i.status <> 'VOID' AND i.billing_month >= date_trunc('month',
current_date) - interval '5 months'
 GROUP BY ROLLUP (c.name, i.invoice_type);
```

Collection-rate ranking (CTE, window functions RANK and LAG)

```
WITH m AS (SELECT complex_name, billing_month, collection_pct FROM
v_monthly_collections)
SELECT billing_month, complex_name, collection_pct,
       RANK() OVER (PARTITION BY billing_month ORDER BY collection_pct DESC)
AS rank_in_month,
       collection_pct - LAG(collection_pct) OVER (PARTITION BY complex_name
ORDER BY billing_month) AS change
  FROM m ORDER BY billing_month DESC, rank_in_month;
```

5.9 Transactions

Every API write runs through `withTransaction()` in `src/lib/db.ts`: `BEGIN`, set `app.user_id`, run statements, `COMMIT`, or `ROLLBACK` on any error. Creating a maintenance request inserts the request and its embedding in one transaction; `sp_terminate_lease` performs four dependent updates atomically; the payment trigger row-locks the invoice (`SELECT ... FOR UPDATE`) so two concurrent payments cannot both pass the balance check.

6. NoSQL / Vector Database Commands

The second database paradigm is a **vector database (pgvector)** that lives inside PostgreSQL, complemented by PostgreSQL full-text search.

6.1 Design summary

Question	Answer
What is embedded	Each maintenance request's `title
Model	`sentence-transformers/all-MiniLM-L6-v2` (ONNX build `Xenova/all-MiniLM-L6-v2`), mean pooling, L2-normalised, 384 dimensions, run locally with transformers.js
Storage	Table `request_embedding` with `vector(384)`, model name and an md5 `source_hash` to detect stale vectors
Index	HNSW with `vector_cosine_ops` (m = 16, ef_construction = 64)
Metric	Cosine distance `<=>`; similarity shown as `1 - distance`
Retrieval modes	Semantic, keyword, and hybrid (Reciprocal Rank Fusion, `score = sum 1/(60 + rank)`)
Used for	Maintenance search box, similar past requests, duplicate detection on creation (similarity above 0.6)

6.2 Setup, storage and index

```
CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE request_embedding (
  request_id  bigint PRIMARY KEY REFERENCES maintenance_request(request_id)
ON DELETE CASCADE,
  model       varchar(80) NOT NULL,
  source_hash char(32)    NOT NULL,   -- md5 of embedded text; mismatch =
stale vector
  embedding   vector(384) NOT NULL,  -- L2-normalised MiniLM embedding
  embedded_at timestamptz NOT NULL DEFAULT now()
);

CREATE INDEX idx_request_embedding_hnsw
  ON request_embedding USING hnsw (embedding vector_cosine_ops)
  WITH (m = 16, ef_construction = 64);
```

6.3 Insert and maintain vectors

The application computes the embedding in Node.js (`src/lib/embeddings.ts`) and inserts it in the same transaction as the request.

```
INSERT INTO request_embedding (request_id, model, source_hash, embedding)
VALUES ($1, 'Xenova/all-MiniLM-L6-v2', md5($2), $3::vector);
```

A trigger (`mr_invalidate_embedding`) deletes the stored vector when the request's title or description is edited, and `npm run db:embed` re-embeds any request whose vector is missing or stale.

6.4 Retrieval queries

Semantic search (nearest neighbours, filtered by the user's access scope)

```
SELECT r.request_id, r.title, 1 - (e.embedding <=> $1::vector) AS similarity
  FROM request_embedding e JOIN maintenance_request r USING (request_id)
 ORDER BY e.embedding <=> $1::vector
 LIMIT 10;
```

Keyword search (full-text, GIN index on `search_tsv`)

```
SELECT request_id, title, ts_rank_cd(search_tsv, q) AS rank
  FROM maintenance_request, websearch_to_tsquery('english', $1) q
 WHERE search_tsv @@ q
 ORDER BY rank DESC LIMIT 10;
```

Hybrid search (vector + full-text fused with RRF)

```
SELECT * FROM fn_hybrid_search('MCB tripping when geyser is on',
$embedding::vector, NULL, 10);
```

Similar past requests (uses the stored vector, no model call)

```
SELECT * FROM fn_similar_requests(123, 5);
```

6.5 Why vector search is needed

The query "water dripping from the roof" matches zero requests with full-text search because no request uses those words together, while semantic search returns five plumbing requests such as "Water seeping through bathroom ceiling". Likewise "insects in the kitchen" returns "Cockroaches in kitchen cabinets" first. Hybrid search keeps exact-term strength for part names and codes ("MCB", "RO membrane").

7. Next.js – Project Tree Structure

```
DBMS_Project/
├── db/
|   ├── 00_extensions.sql          pgvector, btree_gist, pg_trgm
|   ├── 01_schema.sql              domains, 19 tables, keys, CHECK and
EXCLUDE constraints
|   ├── 02_indexes.sql             B-tree, partial, expression, GIN, trigram,
HNSW
|   ├── 03_functions_triggers.sql  business-rule triggers, history and audit
triggers
|   ├── 04_procedures.sql          billing, late fees, payments,
sp_terminate_lease, vector search functions
|   ├── 05_views.sql               7 views + materialized view for dashboard
KPIs
|   ├── 06_migration_v2.sql        ALTER / DROP schema evolution
|   ├── 07_comments.sql            COMMENT ON documentation
|   └── 99_teardown.sql            DROP everything
├── scripts/
```

```
|   ├── seed.ts, embed.ts          seed data and embedding backfill
|   └── db-setup.ts                applies db/*.sql in order
├── src/
|   ├── proxy.ts                   redirects signed-out page requests to
/login
|   ├── lib/                       db pool + transactions, auth, API wrapper,
scoping, embeddings, reports
|   ├── app/
|   |   ├── api/                   REST endpoints (30 route files, 38
handlers; only code that talks to the DB)
|   |   ├── (app)/                 pages: dashboard, complexes, units,
leases, billing, parking,
|   |   |                          maintenance, people, reports, audit
|   |   └── globals.css            design tokens (Tailwind v4 @theme)
|   └── components/                app shell and UI primitives
├── tests/
|   ├── db/                        integrity + vector tests (rolled-back
transactions)
|   └── api/                       end-to-end API tests
├── docs/                          report, data dictionary, schema, diagrams,
screenshots, test results
├── public/
├── .env.example
├── docker-compose.yml
├── next.config.ts
├── package.json
├── tsconfig.json
├── eslint.config.mjs
├── postcss.config.mjs
├── vitest.config.mts
└── README.md
```

Architecture. The browser calls `/api/*` with `fetch` and never touches the database. Each route handler is wrapped by `api()` in `src/lib/api.ts`, which verifies the HttpOnly JWT cookie, re-checks that the account is active, enforces allowed roles, validates input with zod, and maps PostgreSQL errors to HTTP codes (unique and EXCLUDE violations to 409, CHECK and trigger violations to 422). `src/lib/scope.ts` holds the row-visibility predicate used by every list query. All SQL is parameterised.

Security. bcrypt (cost 10) with a database CHECK that rejects non-bcrypt values; HS256 JWT with 8-hour expiry in an `HttpOnly; SameSite=Lax` cookie; 5 failed logins per email per 15 minutes; CSRF protection through SameSite plus JSON-only content type; no hard-

coded credentials (`DATABASE_URL` and `JWT_SECRET` come from `.env.local`).

8. Sample Screenshots of Application Code and Demo

Screenshots of the running application are in the repository under `docs/screenshots/`. Insert each image below its caption before submission.

8.1 Application demo

No.	Screen	File
1	Login with demo accounts	`01-login.png`
2	Admin dashboard: KPIs, billed vs collected, occupancy, SLA	`02-admin-dashboard.png`
3	Complexes	`03-complexes.png`
4	Floor-by-floor unit map	`04-complex-layout.png`
5	Units with filters	`05-units.png`
6	Unit detail	`06-unit-detail.png`
7	Leases	`07-leases.png`
8	New lease form	`08-new-lease-modal.png`
9	Invoices with totals and filters	`09-billing-invoices.png`
10	Monthly billing run (stored function)	`10-billing-run-modal.png`
11	Parking slot map and utilisation	`11-parking.png`
12	Hybrid search: "water dripping from the roof"	`12-semantic-search.png`
13	Semantic search: "lift is not moving"	`12b-semantic-search-lift.png`
14	Request detail with similar past requests	`13-request-detail-similar.png`
15	People (typo-tolerant search)	`14-people.png`
16	Reports with SQL shown	`15-reports-defaulters.png`
17	ROLLUP report	`16-reports-rollup.png`
18	Audit log written by triggers	`17-audit-log.png`

No.	Screen	File
19	Tenant dashboard	`18-tenant-dashboard.png`
20	Tenant paying an invoice	`19-tenant-pay-modal.png`
21	Tenant raising a request	`20-tenant-new-request.png`
22	Tenant parking	`21-tenant-parking.png`
23	Owner dashboard	`22-owner-dashboard.png`
24	Manager dashboard scoped to one complex	`23-manager-dashboard-scoped.png`

Base path: https://github.com/Kundhan73/DBMS_Project/tree/main/docs/screenshots

8.2 Application code (add your own screenshots)

No.	Code to capture	File
C1	Overlap-proof lease constraint and payment triggers	`db/01_schema.sql`, `db/03_functions_triggers.sql`
C2	Billing function and lease termination procedure	`db/04_procedures.sql`
C3	HNSW index and `request_embedding` table	`db/01_schema.sql`, `db/02_indexes.sql`
C4	Embedding pipeline	`src/lib/embeddings.ts`
C5	Transaction helper	`src/lib/db.ts`
C6	API wrapper with auth, roles and zod validation	`src/lib/api.ts`
C7	Reports with their SQL	`src/lib/reports.ts`

8.3 Demo accounts

Role	Email
Admin	admin@aptmgr.in

Role	Email
Manager (Lakeview + Sunrise)	ravi.manager@aptmgr.in
Manager (Palm Grove only)	priya.manager@aptmgr.in
Owner (2 flats)	owner@example.com
Tenant	tenant@example.com

9. Evaluation Rubrics – Completion Status

The table maps each project guideline requirement to where it is met and its status, based on the repository documentation.

Requirement	Where it is met	Status
Unique, non-CRUD real-world problem	Billing engine, analytics, semantic search	Completed
2-member team with defined responsibilities	Team and contributions	Completed
SQL (mandatory) + second database paradigm	PostgreSQL + pgvector, plus full-text search	Completed
ER diagram to relational schema (entities, PK, FK, relationships, constraints)	Conceptual and logical ER, `docs/RELATIONAL_SCHEMA.md`, data dictionary	Completed
Vector component: data model, index, embeddings, metadata, retrieval	Section 6; HNSW cosine index; semantic, keyword, hybrid	Completed
DDL: CREATE, ALTER, DROP, keys, constraints, indexes, views	Section 5.1 to 5.4, 5.7	Completed
DML: INSERT, UPDATE, DELETE, SELECT with JOIN, GROUP BY, HAVING, subqueries	Section 5.8 and application endpoints	Completed

Requirement	Where it is met	Status
At least four advanced concepts	Transactions, triggers, stored procedures, views and materialized view, partial/GIN/HNSW indexes, window functions, ROLLUP, EXCLUDE constraints	Completed
Next.js interface with forms, tables, search, filters, dashboards	Section 8	Completed
Backend/API layer; no hard-coded credentials	`src/app/api`, `.env.example`	Completed
Authentication with at least 2 roles	4 roles: Admin, Manager, Owner, Tenant	Completed
Validation at database level; integrity and consistency	CHECK, EXCLUDE, domains, triggers, transactions	Completed
Security; no plain-text passwords	bcrypt, HttpOnly JWT, parameterised SQL	Completed
Test cases	75 automated tests (37 database, 12 vector, 26 API), all passing per `docs/TEST_RESULTS.md`	Completed
Justification of database choice per data type	Section 4	Completed
GitHub repository with README	Repository root	Completed
Documentation	Full report, data dictionary, relational schema, diagrams	Completed

Known limitations (out of scope by design): real payment gateway, email/SMS notifications, visitor management, amenity booking, accounting ledgers and GST invoicing.

Future enhancements: PostgreSQL row-level security keyed on `app.user_id`; scheduled jobs with `pg_cron`; payment gateway in test mode; k-NN suggestion of request category and priority; notifications; table partitioning of `invoice` and `payment`; multilingual embeddings.